

INTELLIGENT CODE REVIEW ASSISTANT

Using Deep Learning & Natural Language Processing

GitHub URL: <https://github.com/nisansalasandu/intelligent-code-review-assistant>

Project Report

Individual Data Science Internship Project

Submitted by:

Ruwan Pathiranage Sanduni Nisansala

Duration: 4 Weeks

Abstract

This report presents the design, implementation, and evaluation of an Intelligent Code Review Assistant — a deep learning-based system that automatically analyses source code and identifies potential bugs, vulnerabilities, and code smells. The system leverages CodeBERT, a pre-trained transformer model fine-tuned on a combined dataset of 39,656 labelled code samples drawn from CodeSearchNet, the Devign vulnerability dataset, and GitHub bug-fix pull requests.

The pipeline comprises four stages implemented across four Jupyter/Google Colab notebooks: (1) data collection and exploratory data analysis, (2) text preprocessing and tokenization, (3) model training with class-weighted fine-tuning, and (4) evaluation with attention-based explainability. The final model achieved a test accuracy of 74.2%, F1-score of 0.646, precision of 0.530, recall of 0.826, and ROC-AUC of 0.852, demonstrating strong capability to detect the majority of real bugs while maintaining a reasonable false-positive rate.

A Streamlit web dashboard was developed to deploy the model, providing real-time code review with confidence scores, token-level attention visualization, and interactive analysis tools. The project demonstrates the practical applicability of transformer-based NLP techniques to software engineering quality assurance tasks.

Keywords: CodeBERT, Bug Detection, Deep Learning, Transformer, NLP, Code Review, Explainability, Streamlit, Fine-tuning

Table of Contents

Abstract 2

Table of Contents 3

1. Introduction 5

 1.1 Background and Motivation 5

 1.2 Project Objectives..... 5

 1.3 Scope and Constraints 5

 1.4 Tools and Environments..... 6

2. Literature Review 7

 2.1 Code Representation Learning 7

 2.2 Transformer Models for Code 7

 2.3 Defect Detection Datasets 7

 2.4 Explainability in Code Models 8

3. Methodology 9

 3.1 Overall Pipeline Architecture 9

 3.2 Model Architecture..... 9

4. Implementation..... 11

 4.1 Environment and Tools 11

 4.2 Notebook 01 — Data Collection & Exploratory Data Analysis..... 11

 4.2.1 CodeSearchNet Dataset 12

 4.2.2 Devign Vulnerability Dataset 12

 4.2.3 GitHub Bug-Fix Pull Requests 12

 4.2.4 Combined Dataset 12

 4.2.5 Exploratory Data Analysis..... 12

 4.3 Notebook 02 — Preprocessing & Tokenisation..... 16

 4.3.1 Text Cleaning 16

 4.3.2 Tokenization 16

 4.3.3 Train/Validation/Test Split 17

 4.3.4 Class Weight Computation..... 18

 4.3.5 Outputs Saved..... 18

 4.4 Notebook 03 — Model Training 19

 4.4.1 Training Configuration 19

 4.4.2 Training Results..... 19

 4.5 Notebook 04 — Evaluation & Explainability 21

 4.5.1 Test Set Evaluation..... 21

 4.5.2 Attention-Based Explainability 22

 4.5.3 SHAP Explainability 24

5. Streamlit Dashboard 25

 5.1 Overview 25

 5.2 Dashboard Features 25

 5.3 Dashboard Screenshots..... 26

6. Results and Discussion 29

 6.1 Model Performance Summary 29

 6.2 Analysis of Results 29

 6.2.1 Recall vs. Precision Trade-off 29

 6.2.2 ROC-AUC vs. F1 Discrepancy 29

 6.2.3 Comparison with Baseline..... 29

 6.2.4 Training Dynamics 30

 6.3 Explainability Analysis 30

 6.4 Limitations..... 30

7. Future Work 32

 7.1 Model Improvements..... 32

 7.2 Explainability 32

 7.3 Deployment 32

 7.4 Evaluation..... 32

8. Conclusion..... 33

Appendix 34

 Appendix A — Project File Structure 34

 Appendix B — Complete List of Saved Figures..... 34

 Appendix C — Running the Project..... 35

 C.1 Environment Setup 35

 C.2 Run Notebooks (in order) 36

 C.3 Launch Dashboard 36

1. Introduction

1.1 Background and Motivation

Software bugs and vulnerabilities represent one of the most significant challenges in modern software development. Studies consistently show that bugs discovered late in the development cycle are exponentially more expensive to fix than those caught early. Traditional code review, a manual process where developers inspect each other's code is time-consuming, inconsistent, and scales poorly with team size and codebase complexity.

Recent advances in Natural Language Processing (NLP), particularly transformer-based language models pre-trained on large code corpora, have opened new possibilities for automated code understanding. Models such as CodeBERT, GraphCodeBERT, and CodeT5 have demonstrated strong performance on tasks including code search, code summarization, and defect detection. This project harnesses these capabilities to build a practical, deployable code review assistant.

1.2 Project Objectives

This internship project was designed to achieve the following objectives:

- ❖ Develop a deep learning model capable of detecting bugs, code smells, and inefficiencies in source code
- ❖ Apply NLP techniques, specifically transformer-based models to understand code semantics
- ❖ Train the model using publicly available datasets including CodeSearchNet and Devign
- ❖ Integrate the trained model into a Streamlit web dashboard providing automated code review suggestions
- ❖ Evaluate model performance using accuracy, precision, recall, F1-score, and ROC-AUC metrics
- ❖ Provide explainability for model predictions using attention weight visualization
- ❖ Document the complete pipeline from data collection to deployment

1.3 Scope and Constraints

The project was completed as a 4-week individual internship task. Given time and computational constraints, the primary programming language focus is Python (for clean code from CodeSearchNet) and C (for labelled vulnerable code from Devign). The model is implemented as a binary classifier (clean vs. buggy) rather than a multi-class categoriser. Explainability is provided through attention-weight analysis rather than gradient-based methods such as Integrated Gradients, which require additional computational resources.

1.4 Tools and Environments

The project utilized two computational environments:

- ❖ **Jupyter Notebook (local):** Notebooks 01 and 02 — data collection, EDA, preprocessing, and tokenization
- ❖ **Google Colab (cloud GPU — NVIDIA T4):** Notebooks 03 and 04 — model training, evaluation, and explainability
- ❖ **Streamlit:** Web dashboard deployment (app/app.py)

This hybrid approach was adopted to leverage local flexibility for data preparation and GPU acceleration for computationally intensive training without incurring cost.

2. Literature Review

2.1 Code Representation Learning

Early approaches to automated code analysis relied on hand-crafted features extracted from abstract syntax trees (ASTs), control flow graphs (CFGs), and program dependency graphs (PDGs). While effective for specific bug patterns, these methods required domain expertise to design and were brittle to syntactic variation.

The introduction of token-level code embeddings, analogous to word embeddings in NLP - enabled more flexible representations. Models such as code2vec and code2seq represented programs as paths in ASTs, capturing structural information while remaining differentiable. However, these approaches still required explicit structural parsing and did not generalize well across languages.

2.2 Transformer Models for Code

The BERT model demonstrated that large-scale pre-training on masked language modelling and next-sentence prediction tasks yields powerful transferable representations for natural language. Subsequent work adapted this paradigm to source code:

- ❖ **CodeBERT:** A bimodal pre-trained model for programming language and natural language, trained on the CodeSearchNet corpus using masked language modelling and replaced token detection. Achieves state-of-the-art performance on code search, code documentation, and defect detection.
- ❖ **GraphCodeBERT:** Extends CodeBERT by incorporating data flow graphs, improving performance on structure-dependent tasks such as code clone detection.
- ❖ **CodeT5:** A sequence-to-sequence model enabling both code understanding and generation tasks, including automated bug fixing.

This project uses CodeBERT as the backbone model due to its strong established baseline on the Devign defect detection benchmark, availability as a pre-trained HuggingFace model, and broad language support including Python and C.

2.3 Defect Detection Datasets

Several benchmark datasets have been established for code defect detection research:

- ❖ **CodeSearchNet:** Contains approximately 2 million (code, documentation) pairs from open-source repositories in 6 programming languages. Used primarily as a source of clean code samples in this project.

- ❖ **Devign:** A dataset of 27,318 C functions labelled as vulnerable or non-vulnerable, constructed from commits in four large open-source projects. Widely used as the standard benchmark for deep-learning-based vulnerability detection.
- ❖ **BigVul / ReVeal:** Larger vulnerability datasets extending Devign, not used in this project due to time constraints.

2.4 Explainability in Code Models

Interpretability of model decisions is critical for practical adoption of automated code review tools. Developers need to understand not only whether code is flagged as buggy, but which specific tokens or constructs triggered the prediction. Approaches explored in the literature include:

- ❖ **Attention visualization:** Extracting attention weights from transformer layers to identify tokens the model focuses on. Simple to implement but not a guaranteed measure of causation.
- ❖ **SHAP (SHapley Additive exPlanations):** Model-agnostic feature attribution that satisfies theoretical axioms of fairness and consistency. More computationally expensive but theoretically grounded.
- ❖ **Integrated Gradients:** Gradient-based attribution method attributing predictions to input token embeddings.

This project implements attention-based explainability (last transformer layer, mean over 12 attention heads, CLS-to-token attention projection) as a lightweight and visually intuitive approach. SHAP was additionally explored in Notebook 04 for comparison.

3. Methodology

3.1 Overall Pipeline Architecture

The project pipeline follows a standard machine learning workflow adapted for deep learning with transformer models. The four-stage pipeline is illustrated below:

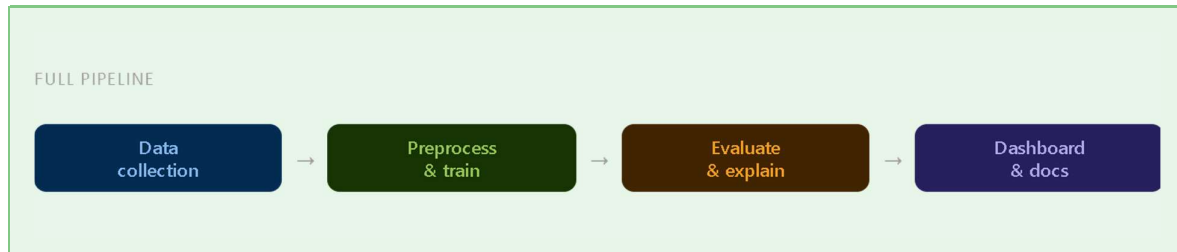


Figure 3.1: End-to-end project pipeline: Data Collection → Preprocessing → Model Training → Evaluation & Deployment

Each stage corresponds to one Jupyter/Colab notebook, with outputs from each stage persisted to disk and consumed by the subsequent stage. This modular design allows independent re-execution of any stage without rerunning the entire pipeline.

3.2 Model Architecture

The classifier is built on top of microsoft/codebert-base, a RoBERTa-style transformer pre-trained on code from six programming languages (Python, Java, JavaScript, PHP, Ruby, Go) using masked language modelling and replaced token detection objectives.

The classification head architecture is:

Input tokens (max 512)

- CodeBERT encoder (12 transformer layers, 768-dim hidden)
- [CLS] token hidden state (768-dim)
- Dropout (p=0.1)
- Linear(768 → 2)
- Softmax → P(clean), P(buggy)

The bottom 6 of 12 encoder layers were frozen during training. Only the top 6 encoder layers and the classification head were updated. This strategy reduces training time by approximately 60% and mitigates overfitting on smaller datasets, as the lower layers capture generic syntactic patterns that transfer well without modification.

Component	Specification
Base model	microsoft/codebert-base (RoBERTa-style)
Vocabulary size	50,265 tokens (WordPiece)
Hidden dimension	768
Attention heads	12
Encoder layers	12 (bottom 6 frozen, top 6 trainable)
Dropout	0.1
Classification head	Linear(768 $\rightarrow$ 2)
Total parameters	124,647,170
Trainable parameters	43,119,362 (34.6%)
Max input length	512 tokens

Table 3.1: Model architecture specifications

4. Implementation

4.1 Environment and Tools

Tool / Library	Version	Purpose
Python	3.14	Core programming language
PyTorch	2.11.0	Deep learning framework
HuggingFace Transformers	5.8.0	CodeBERT model & tokeniser
HuggingFace Datasets	4.8.5	CodeSearchNet & Devign loading
Scikit-learn	1.8.0	Metrics, class weights, splits
Streamlit	1.57.0	Web dashboard deployment
SHAP	0.51.0	Explainability (Notebook 04)
PyGitHub	2.9.1	GitHub API data collection
Matplotlib / Seaborn	3.10.9/0.13.2	Visualisation
Pandas / NumPy	2.3.3/ 2.4.4	Data manipulation
Google Colab (T4 GPU)	—	Notebooks 03 & 04 training
Jupyter Notebook (local)	7.5.6	Notebooks 01 & 02

Table 4.1: Tools and libraries used

4.2 Notebook 01 — Data Collection & Exploratory Data Analysis

Environment: Jupyter Notebook (local machine)

This notebook handles the acquisition and initial exploration of all training data from three sources:

4.2.1 CodeSearchNet Dataset

The Python subset of CodeSearchNet was loaded directly from HuggingFace Hub using the datasets library. A sample of 15,000 functions was extracted from the training split, assigned label 0 (clean), and retained for the final combined dataset. The `func_code_string` column was used as the code input; `func_documentation_string` was retained as metadata.

4.2.2 Devign Vulnerability Dataset

The Devign dataset was loaded via the `google/code_x_glue_cc_defect_detection` HuggingFace repository. All three splits (train, validation, test) were combined, yielding 27,318 C functions with binary vulnerability labels. A critical preprocessing fix was applied: Devign's target column returns Python boolean values (True/False) rather than integers (1/0), which caused silent label corruption when concatenated with other sources. An explicit `.astype(int)` cast was applied to resolve this.

4.2.3 GitHub Bug-Fix Pull Requests

The PyGitHub library was used to scrape Python repositories with more than 200 stars. The scraper employed two strategies: (1) searching for issues labelled 'bug', 'bugfix', 'fix', or 'defect' that were linked to merged pull requests, and (2) a fallback scan of PR titles for bug-related keywords. Git diff patches from changed .py files were parsed to extract the removed lines (the buggy code before the fix) using a custom patch extraction function.

4.2.4 Combined Dataset

Source	Language	Label	Samples
CodeSearchNet	Python	Clean (0)	~14,700
Devign	C	Mixed (0/1)	~27,318
GitHub PRs	Python	Buggy (1)	~6
Total	—	—	39,656

Table 4.2: Final combined dataset composition

4.2.5 Exploratory Data Analysis

The following EDA visualizations were produced and saved to `reports/figures/`:

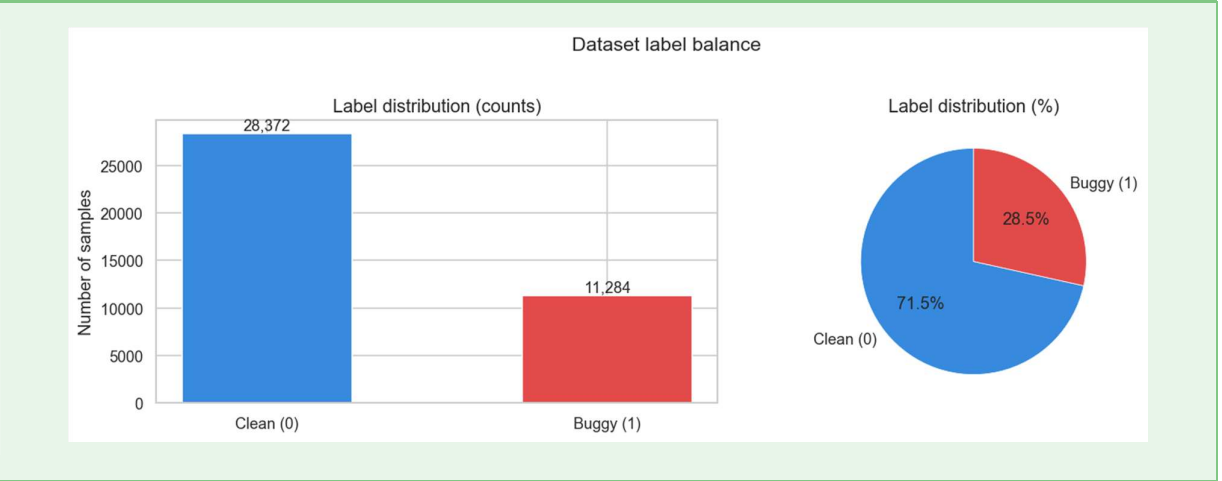


Figure 4.1: Label distribution — count and percentage (clean vs. buggy)

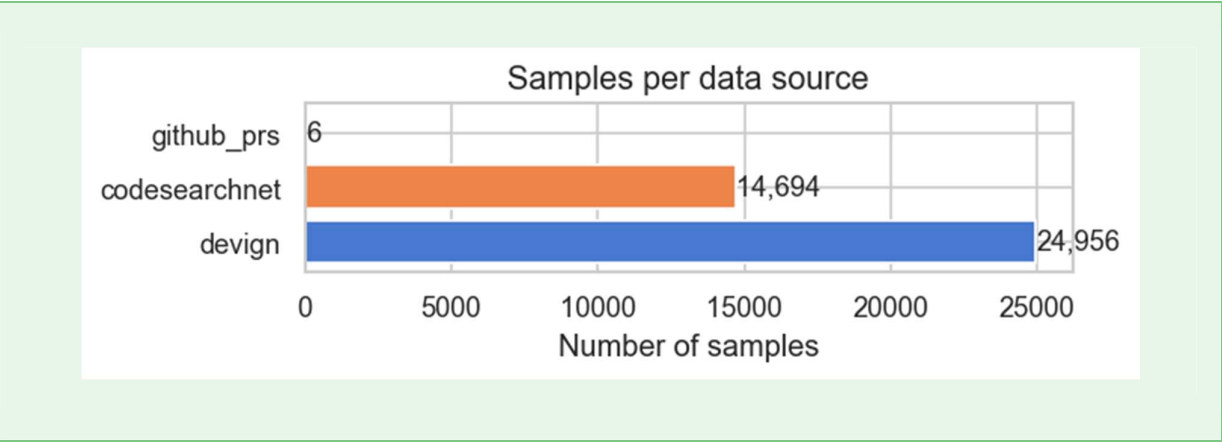


Figure 4.2: Sample count by data source

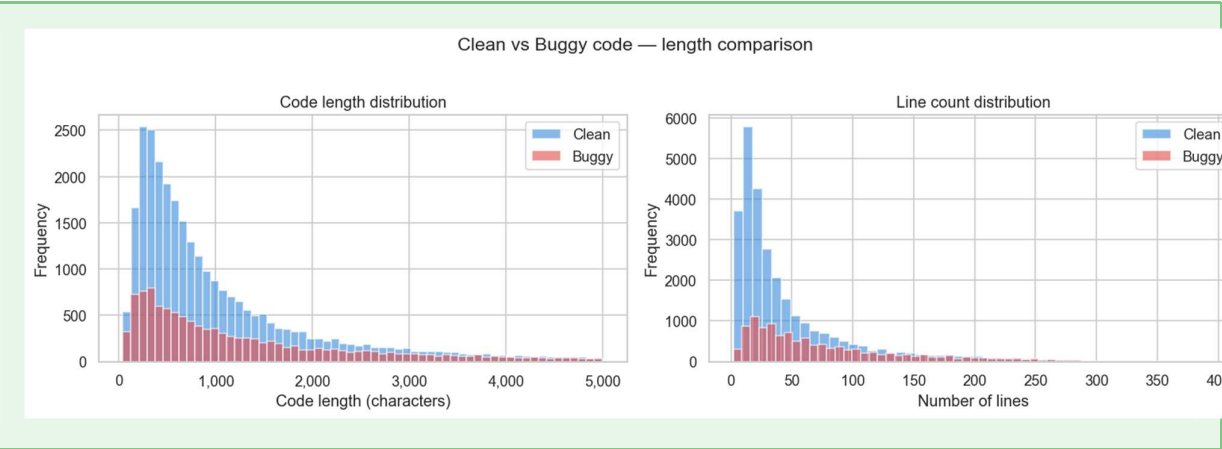


Figure 4.3: Code length distribution — clean vs. buggy comparison

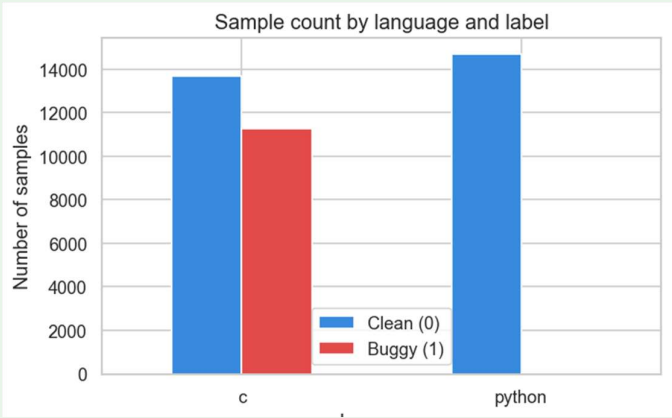


Figure 4.4: Sample count by programming language and label

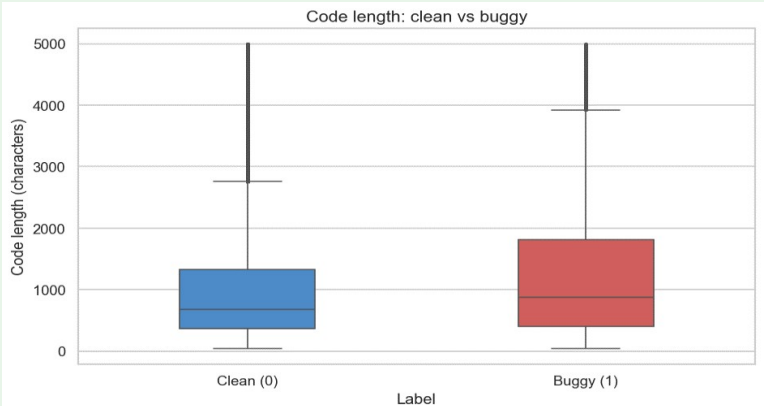


Figure 4.5: Code length boxplot — clean vs. buggy

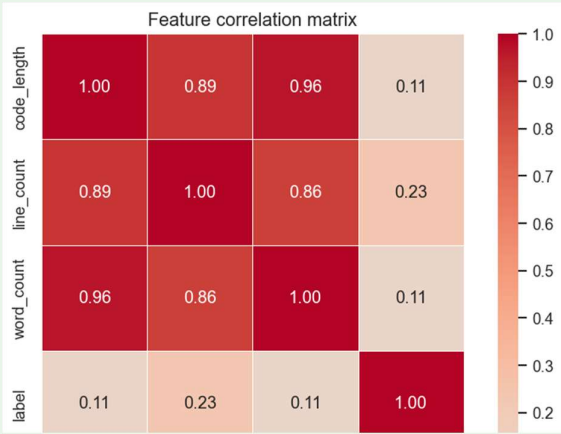


Figure 4.6: Feature correlation heatmap (code_length, line_count, word_count, label)

Key EDA findings:

- ❖ The dataset exhibits a 2.5:1 class imbalance (clean:buggy), handled via weighted cross-entropy loss during training
- ❖ Buggy C code (Devign) tends to be longer on average than clean Python code (CodeSearchNet), due to language verbosity differences
- ❖ Code length and line count are mildly positively correlated with the buggy label (correlation ~ 0.3)
- ❖ GitHub PR scraping yielded very few Python bug samples due to repository label inconsistencies — Devign remains the primary source of buggy samples

4.3 Notebook 02 — Preprocessing & Tokenisation

Environment: Jupyter Notebook (local machine)

This notebook transforms the raw combined dataset into tokenized PyTorch tensors ready for model training.

4.3.1 Text Cleaning

A minimal cleaning function was applied to all code samples:

- ❖ Remove non-UTF-8 bytes (encode/decode with errors='ignore')
- ❖ Remove null bytes (\x00) common in C code
- ❖ Normalize line endings (Windows \r\n to Unix \n)
- ❖ Collapse three or more consecutive blank lines to two
- ❖ Strip leading and trailing whitespace
- ❖ Cap at 5,000 characters to prevent tokeniser memory spikes

Deliberate non-interventions: comments were retained (contain semantic bug context), string literals were preserved, and identifier names were not altered (would break CodeBERT's sub-word alignment).

4.3.2 Tokenization

The AutoTokenizer from microsoft/codebert-base (WordPiece, vocabulary size 50,265) was used with the following parameters:

- ❖ **max_length:** 512 (CodeBERT's hard limit)
- ❖ **padding:** 'max_length' (pad shorter sequences with <pad> tokens)
- ❖ **truncation:** True (cut sequences exceeding 512 tokens from the right)
- ❖ **return_attention_mask:** True (1 for real tokens, 0 for padding)

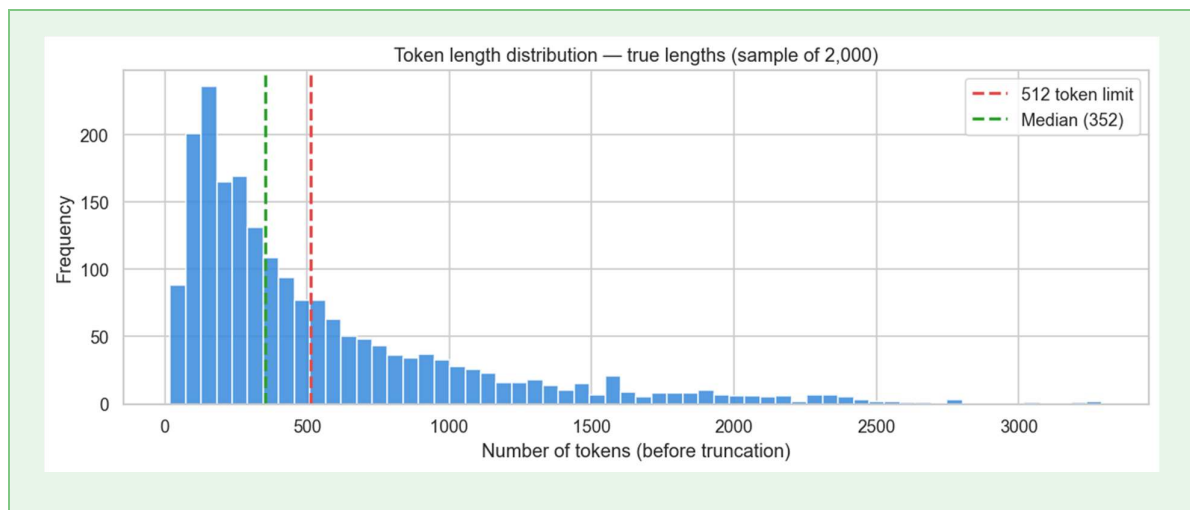


Figure 4.7: Token length distribution before truncation (sample of 2,000)

4.3.3 Train/Validation/Test Split

A stratified split was applied to preserve the class ratio across all three sets:

Split	Samples	Clean (0)	Buggy (1)	Fraction
Train	27,758	19,860	7,898	70%
Validation	5,949	4,256	1,693	15%
Test	5,949	4,256	1,693	15%

Table 4.3: Dataset split statistics

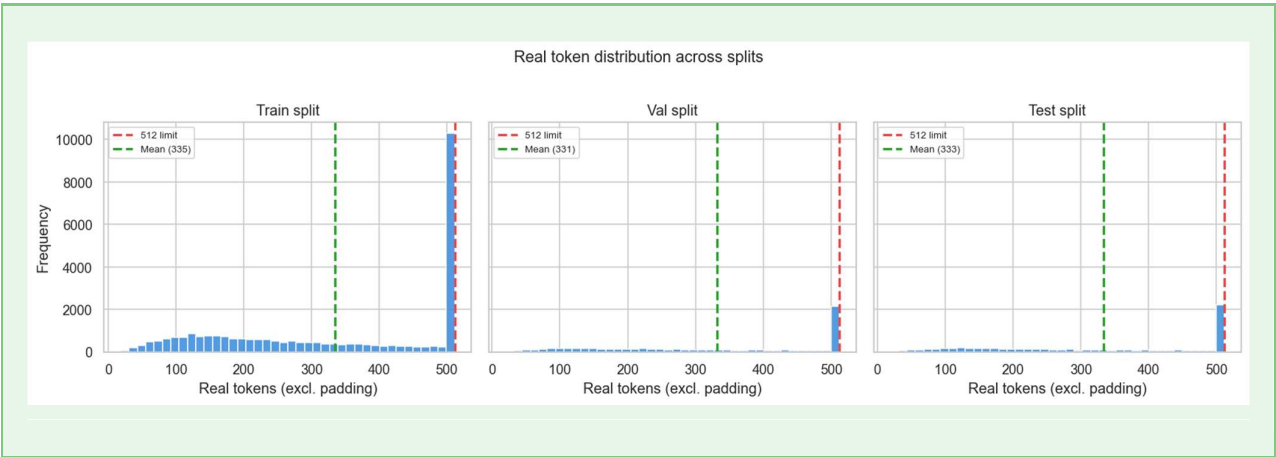


Figure 4.8: Real token distribution across train/validation/test splits

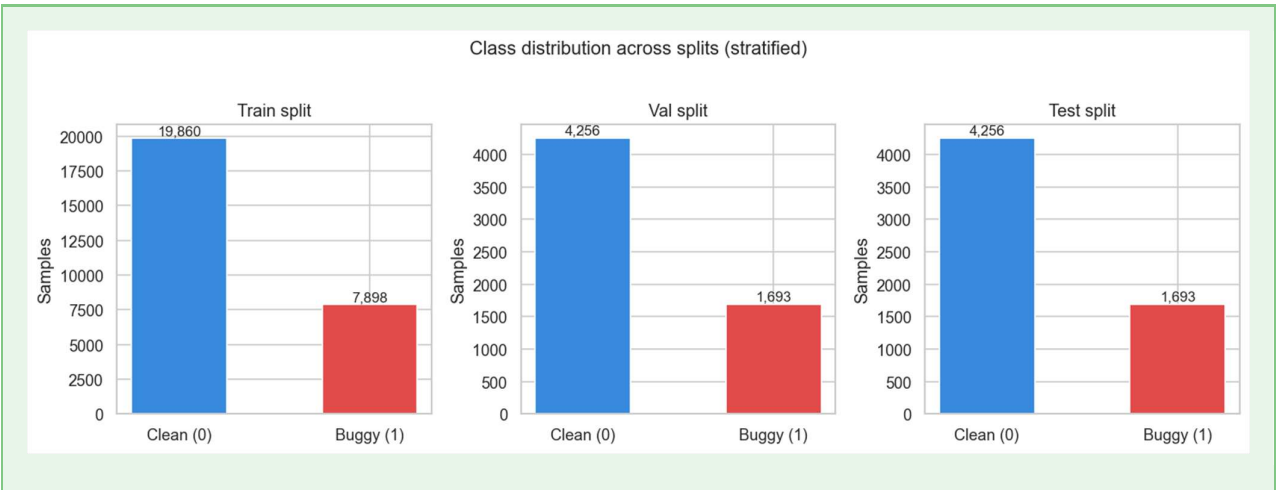


Figure 4.9: Class distribution across splits (stratified)

4.3.4 Class Weight Computation

Class weights were computed from the training set using sklearn's `compute_class_weight` with `class_weight='balanced'`:

- **Weight for class 0 (clean):** 0.6988
- **Weight for class 1 (buggy):** 1.7573

The buggy class receives a weight 2.51x higher than clean, penalizing the model more heavily for missing actual bugs. These weights are passed to the `CrossEntropyLoss` function during training.

4.3.5 Outputs Saved

- `data/processed/train.pt` - PyTorch Dataset (27,758 samples)
- `data/processed/val.pt` - PyTorch Dataset (5,949 samples)
- `data/processed/test.pt` - PyTorch Dataset (5,949 samples)
- `data/processed/class_weights.npy` - numpy array [0.6988, 1.7573]
- `data/processed/preprocessing_config.pkl` - tokeniser and split configuration

4.4 Notebook 03 — Model Training

Environment: Google Colab (NVIDIA T4 GPU)

Training was performed on Google Colab to leverage GPU acceleration. The tokenised .pt datasets were uploaded to Google Drive and mounted in Colab for direct access.

4.4.1 Training Configuration

Hyperparameter	Value	Rationale
Optimizer	AdamW	Standard for transformer fine-tuning; decoupled weight decay
Learning rate	2e-5	Standard CodeBERT fine-tuning rate
Batch size	16	Fits T4 16GB VRAM; effective batch 16×4=64 with gradient accumulation
Gradient accumulation	4 steps	Simulates larger batch without memory overhead
Epochs	6 (early stopping)	Early stopping on validation F1 with patience=2
Warmup ratio	0.1	10% of total steps for linear learning rate warmup
Scheduler	Linear warmup + decay	Prevents large gradient updates early in training
Loss function	Weighted CrossEntropyLoss	Class weights [0.699, 1.757] address imbalance
Frozen layers	Bottom 6 of 12	Reduces training time ~60%, limits overfitting
Dropout	0.1	Applied before classification head

Table 4.4: Training hyperparameters

4.4.2 Training Results

The model was trained for 6 epochs. Best model checkpoint was saved at epoch 3 based on validation F1 score.

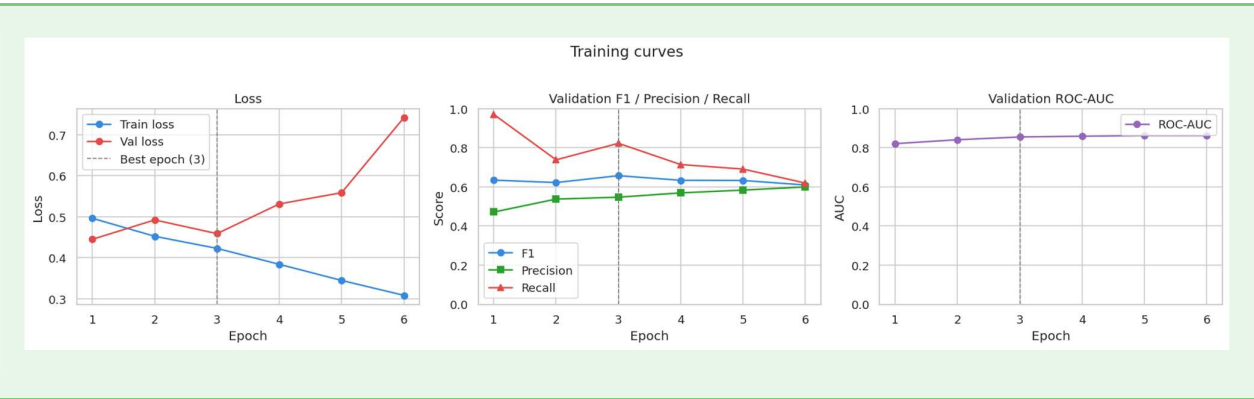


Figure 4.10: Training curves — loss, validation F1, precision/recall, ROC-AUC over 6 epochs

Epoch	Train Loss	Val Loss	Val Accuracy	Val Precision	Val Recall	Val F1	Val ROC-AUC
1	0.4959	0.4448	68.2%	0.4713	0.9705	0.6345	0.8209
2	0.4522	0.4921	74.5%	0.5376	0.7383	0.6222	0.8410
3 *	0.4227	0.4586	75.6%	0.5471	0.8228	0.6572	0.8552
4	0.3838	0.5310	76.5%	0.5695	0.7141	0.6336	0.8593
5	0.3445	0.5585	77.2%	0.5835	0.6911	0.6328	0.8624
6	0.3082	0.7416	77.4%	0.5998	0.6196	0.6095	0.8623

Table 4.5: Training metrics per epoch (*= best checkpoint saved)

Observation: Training loss continues to decrease through all 6 epochs, while validation loss increases after epoch 3, a classic sign of overfitting. Early stopping at epoch 3 prevents the model from memorizing training patterns at the expense of generalization. The high recall (0.97) in epoch 1 and declining recall with increasing precision across epochs reflects the model's gradual shift from a recall-biased to more balanced predictor.

4.5 Notebook 04 — Evaluation & Explainability

Environment: Google Colab (NVIDIA T4 GPU)

4.5.1 Test Set Evaluation

The best checkpoint (epoch 3) was evaluated on the 5,949-sample held-out test set:

Metric	Score	Interpretation
Accuracy	74.23%	Correct predictions on 74.2% of all test samples
Precision	0.5303	53.0% of samples flagged as buggy are truly buggy
Recall	0.8263	82.6% of all actual bugs are successfully detected
F1 Score	0.6460	Harmonic mean of precision and recall
ROC-AUC	0.8519	Strong discrimination between clean and buggy code

Table 4.6: Final test set performance metrics

The precision-recall trade-off reflects the class-weighted training strategy: the model prioritises catching bugs (high recall) at the cost of some false positives (moderate precision). For a code review use case, high recall is generally preferred - it is better to flag a few clean snippets than to miss real bugs.

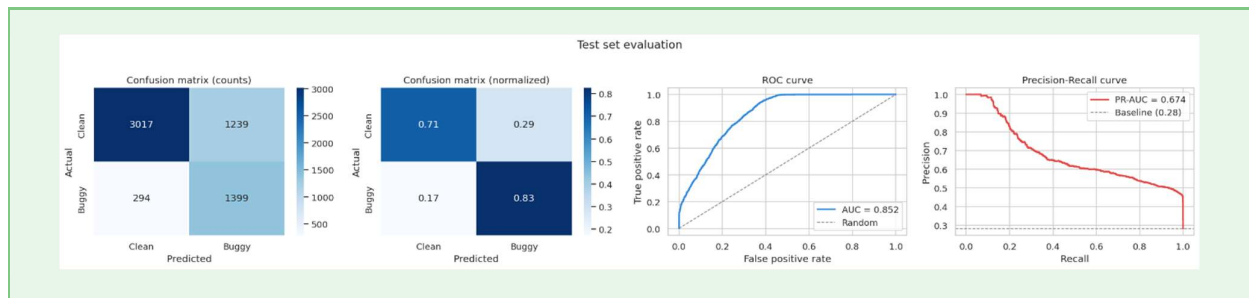


Figure 4.11: Test set evaluation — confusion matrix and ROC curve

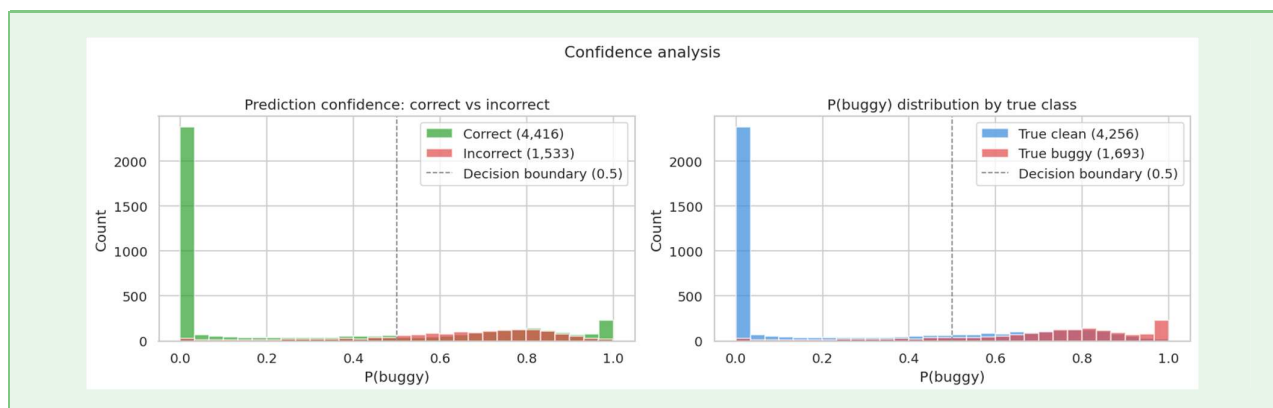


Figure 4.12: Model confidence distribution — true labels coloured

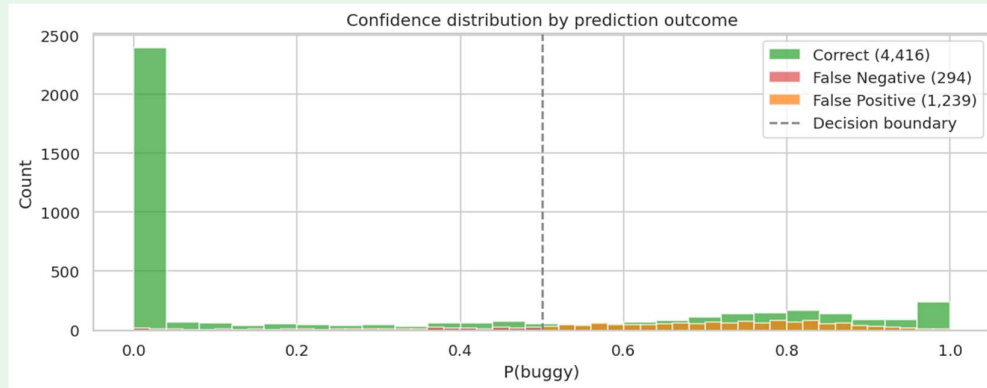


Figure 4.13: Error analysis — false positive and false negative patterns

4.5.2 Attention-Based Explainability

Token-level explainability was implemented using CodeBERT's attention weights. For each prediction, the following procedure is applied:

1. Extract all attention weight matrices from all 12 transformer layers
2. Select the last layer (layer 11) — the most task-specific representations
3. Average attention weights across all 12 attention heads
4. Extract the [CLS]-to-all-tokens row (how much the sequence representation attends to each token)
5. Normalize weights to [0, 1] range
6. Render token highlights: red intensity for buggy predictions, green for clean

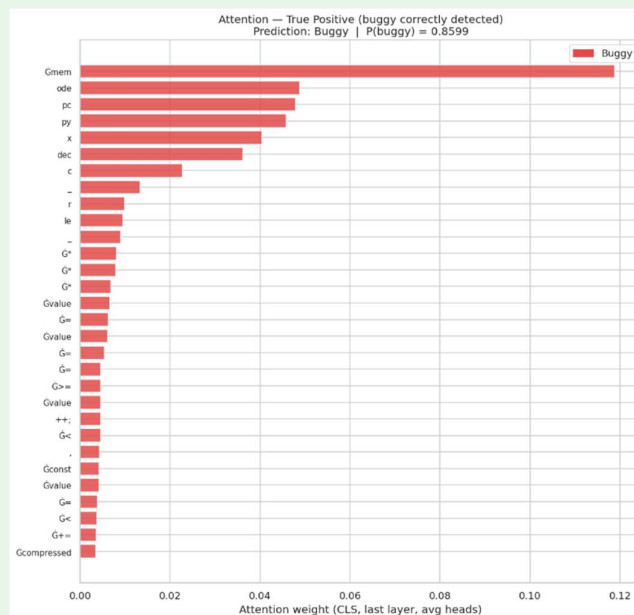


Figure 4.14: Attention visualization — true positive (correctly identified bug)

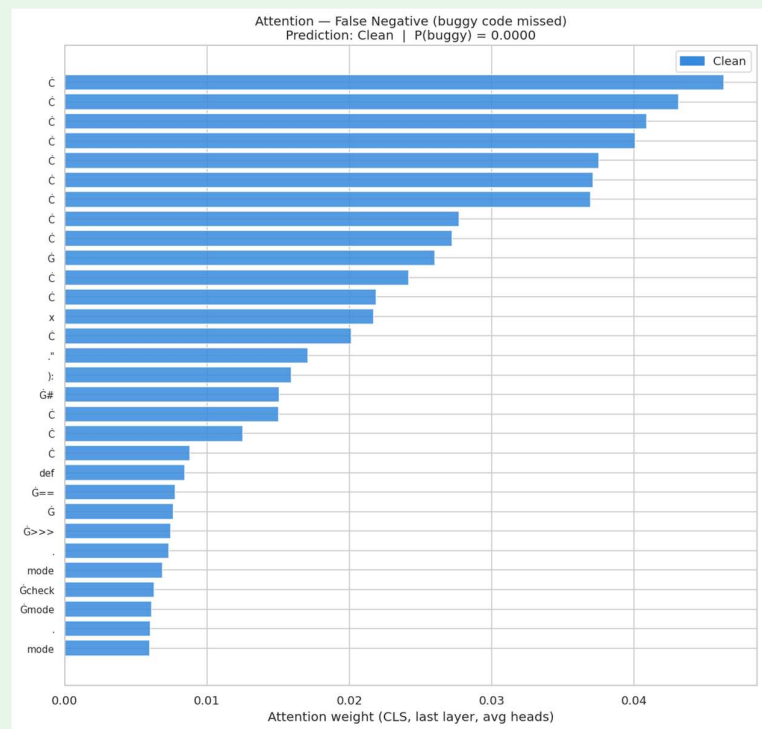


Figure 4.15: Attention visualization — false negative (missed bug)

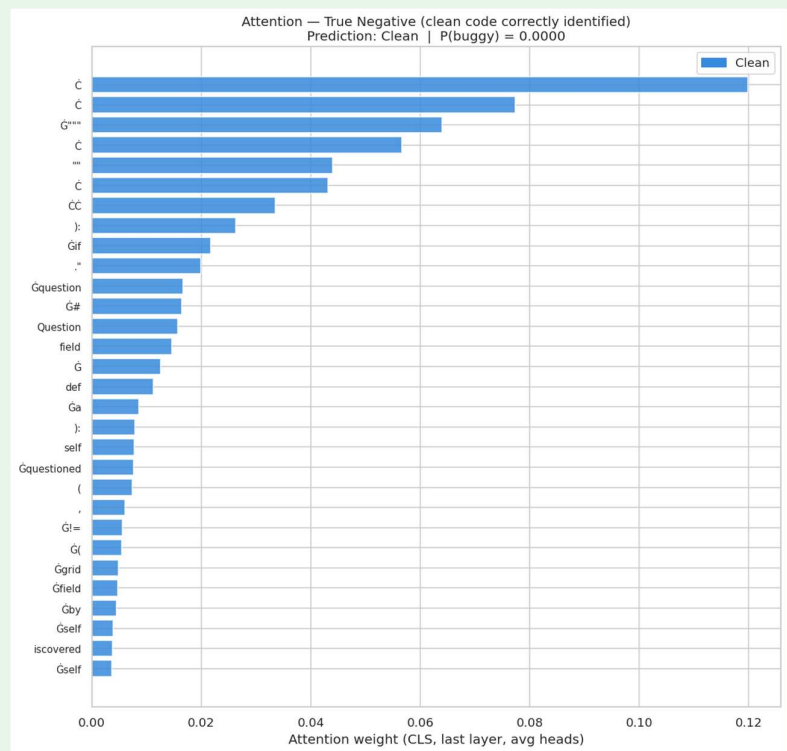


Figure 4.16: Attention visualization — true negative (correctly identified clean code)

4.5.3 SHAP Explainability

SHAP (SHapley Additive exPlanations) was additionally applied to a sample of predictions to provide model-agnostic feature attribution. SHAP values represent each token's marginal contribution to the prediction probability, satisfying consistency and efficiency axioms that attention weights do not guarantee.

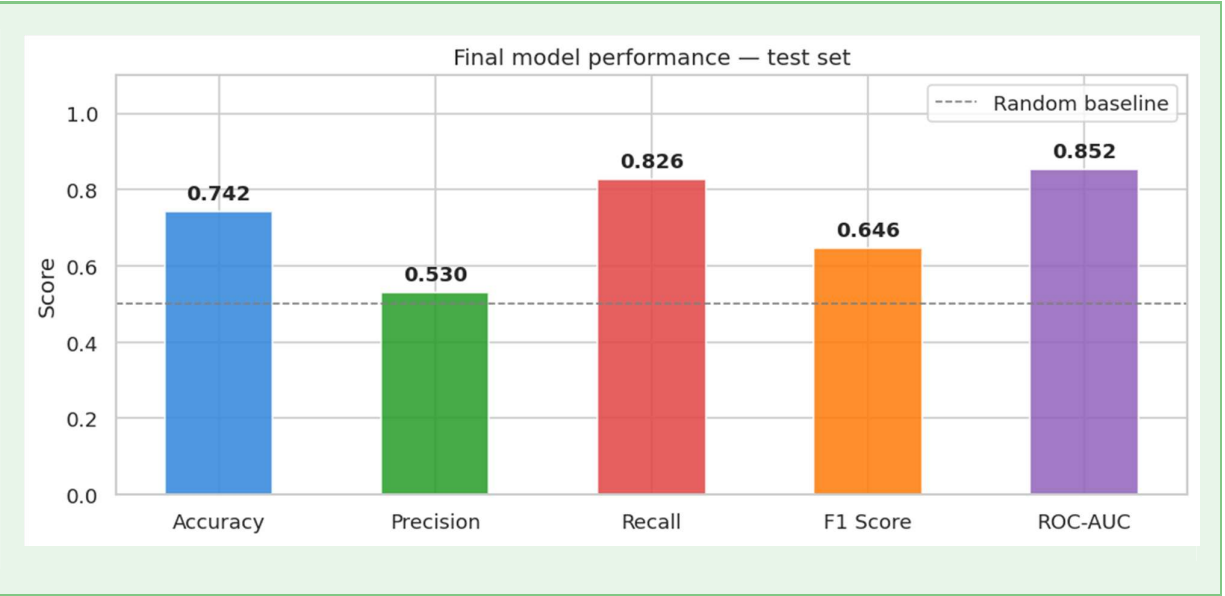


Figure 4.17: Final metrics summary dashboard

5. Streamlit Dashboard

5.1 Overview

A Streamlit web application (app/app.py) was developed to deploy the trained model as an interactive code review tool. The dashboard provides real-time inference, confidence visualization, and attention-based explainability in a user-friendly interface accessible via a web browser.

To launch the dashboard:

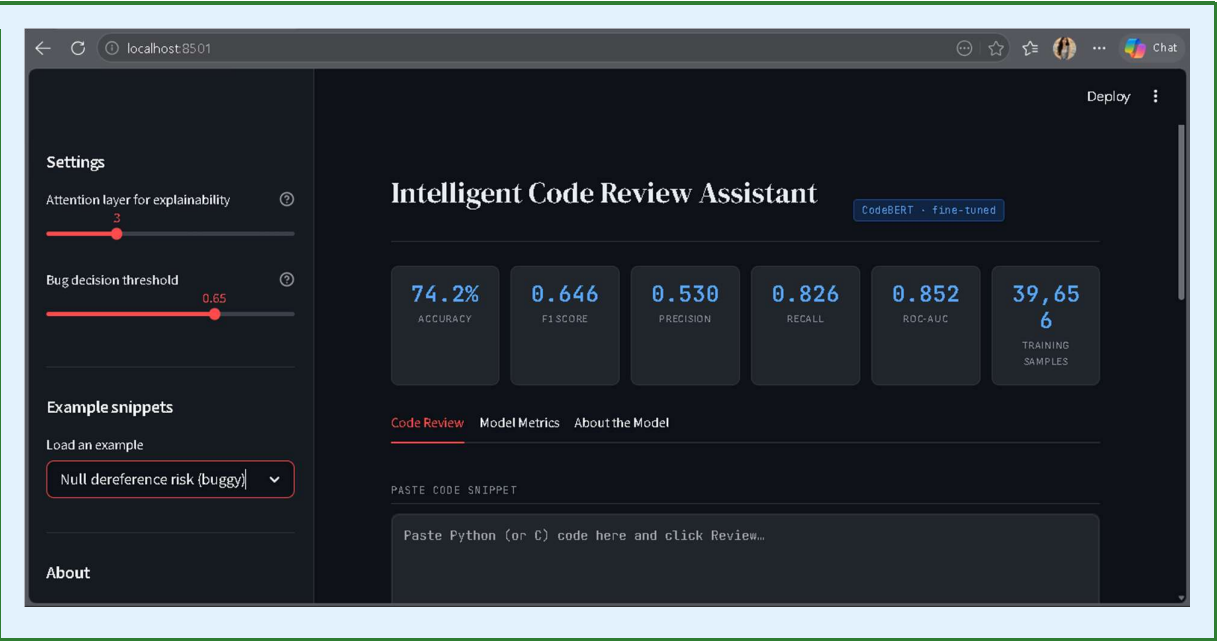
```
streamlit run app/app.py
```

5.2 Dashboard Features

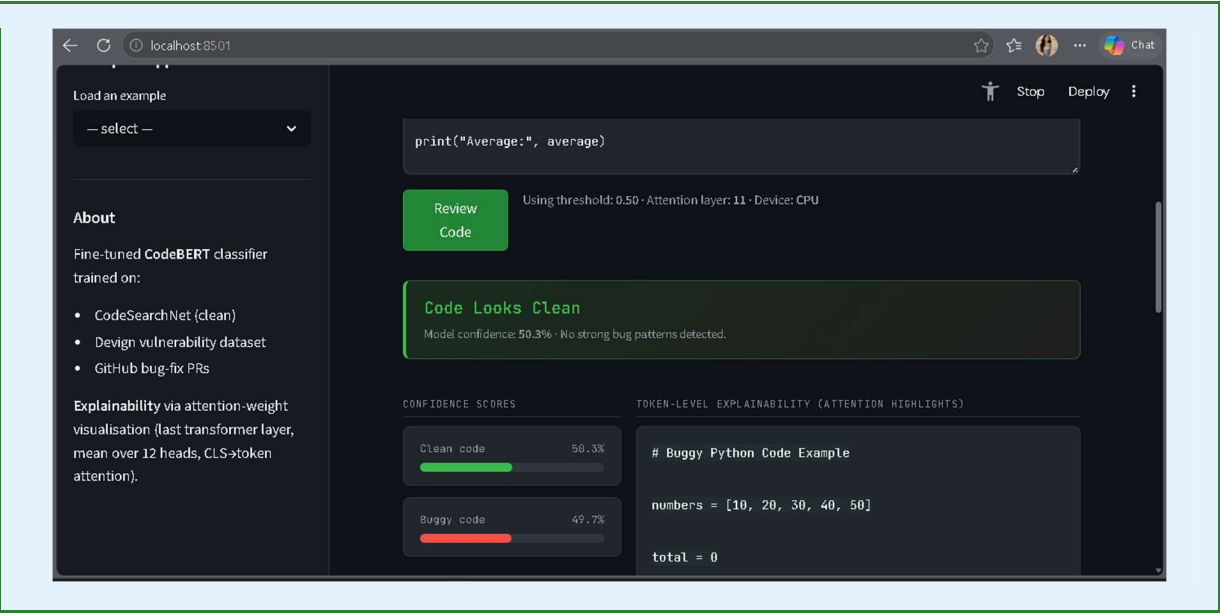
Tab / Section	Feature	Description
Code Review	Code input	Syntax-highlighted text area for pasting code snippets
Code Review	Verdict banner	Red/green banner indicating buggy or clean verdict
Code Review	Confidence bars	Visual probability bars for P(clean) and P(buggy)
Code Review	Token highlight	Color-coded attention visualization of input tokens
Code Review	Top tokens table	Top 15 most-attended tokens with scores
Code Review	Raw JSON output	Full model output including probabilities and metadata
Code Review	Example snippets	5 pre-loaded code examples (buggy and clean)
Sidebar	Decision threshold	Adjustable probability threshold (default 0.50)
Sidebar	Attention layer	Selectable transformer layer for explainability (0-11)
Model Metrics	Metrics table	Accuracy, precision, recall, F1, ROC-AUC
Model Metrics	Training curves	Loss, F1, precision/recall, ROC-AUC over epochs
Model Metrics	Confusion matrix	Predicted vs. actual label breakdown
Model Metrics	Confidence dist.	Histogram of model confidence by true label
About	Architecture	Model architecture, dataset, and pipeline documentation

Table 5.1: Dashboard features and descriptions

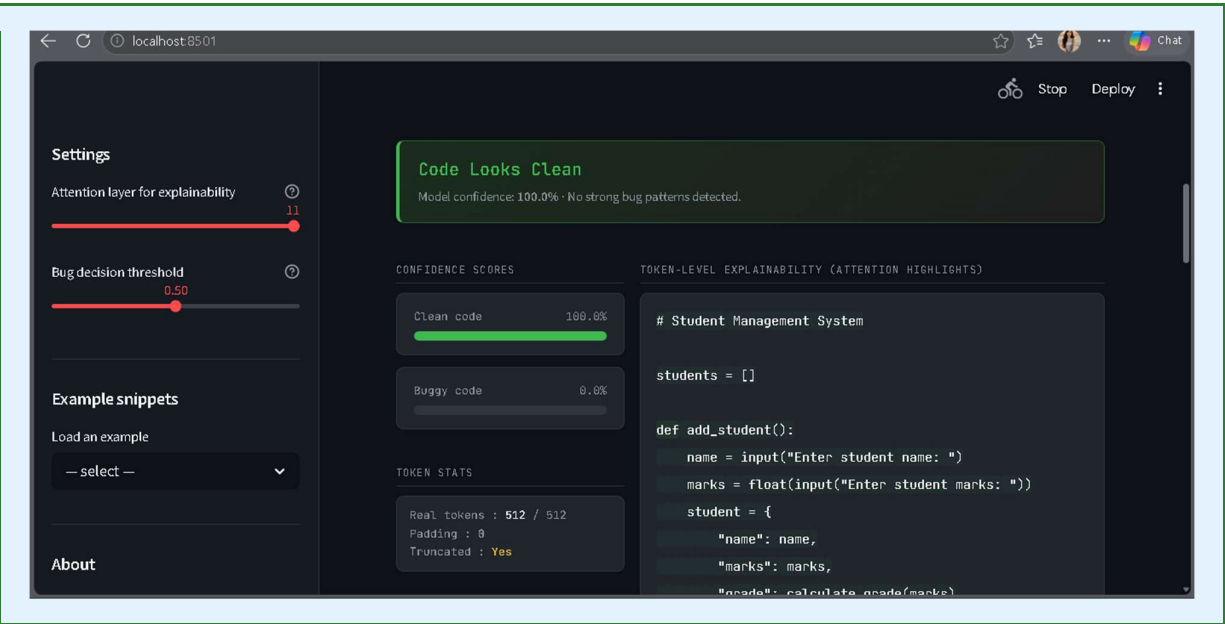
5.3 Dashboard Screenshots



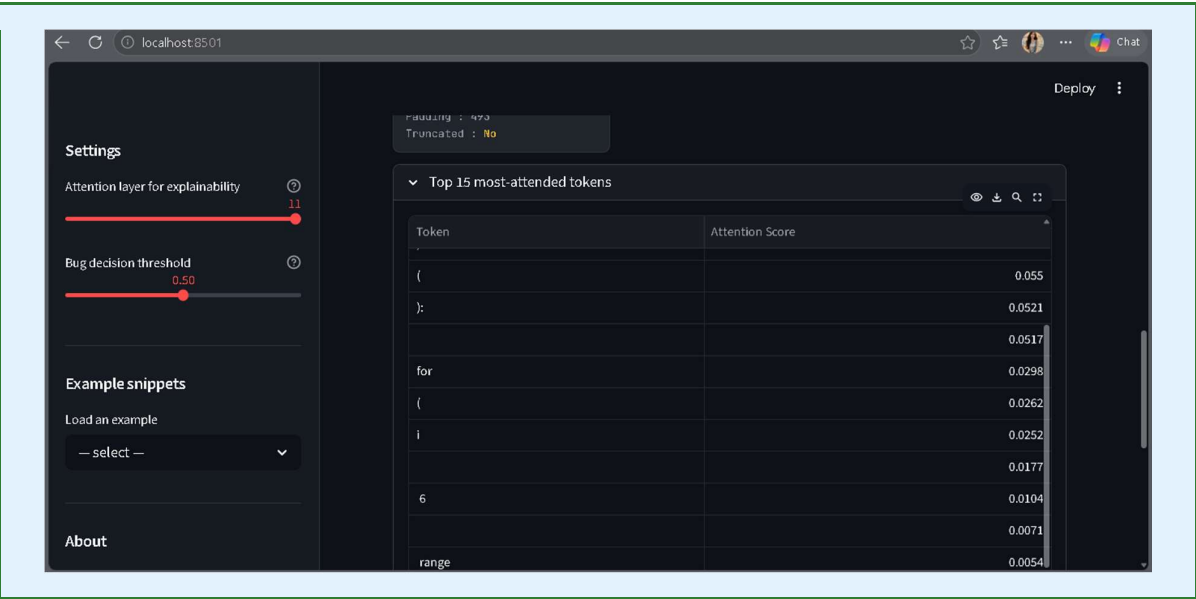
Screenshot 1: Main Code Review tab — overall layout and header metrics



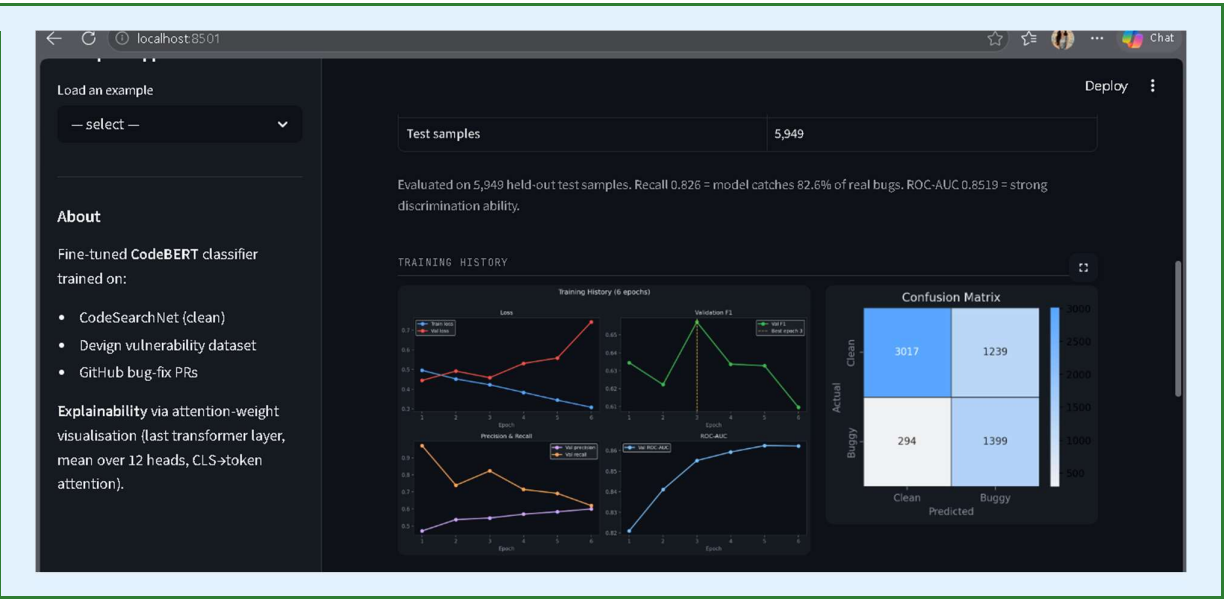
Screenshot 2: Bug detected — verdict banner, confidence bars, and token attention highlights (buggy code example)



Screenshot 3: Clean code — verdict banner and confidence visualization (clean code example)



Screenshot 4: Top 15 most-attended tokens expanded view



Screenshot 5: Model Metrics tab — training curves and confusion matrix

6. Results and Discussion

6.1 Model Performance Summary

The final CodeBERT classifier achieved the following performance on the held-out test set of 5,949 samples:

Metric	Value	Benchmark Context
Accuracy	74.23%	Baseline (always predict majority): ~71.6%
Precision (buggy)	0.5303	~53% of flagged code truly contains bugs
Recall (buggy)	0.8263	Catches 82.6% of all real bugs
F1 Score	0.6460	Devign SOTA (GraphCodeBERT): ~0.73
ROC-AUC	0.8519	Strong; 0.85+ considered excellent for code defect detection
Best Val F1	0.6572	Achieved at epoch 3 (of 6)

Table 6.1: Final model performance vs. benchmarks

6.2 Analysis of Results

6.2.1 Recall vs. Precision Trade-off

The model exhibits higher recall (0.826) than precision (0.530). This is a deliberate outcome of the class-weighted training strategy, which assigns 2.51x higher loss to missed bugs than to false positives. For a practical code review assistant, this trade-off is appropriate: a developer would prefer the tool to flag suspicious code for manual review (accepting some false alarms) rather than silently miss real vulnerabilities.

6.2.2 ROC-AUC vs. F1 Discrepancy

The model achieves ROC-AUC of 0.852, indicating strong discriminative ability — the model is substantially better than random at separating clean from buggy code. However, the F1 score of 0.646 is more moderate, reflecting the imprecision of the binary decision boundary at threshold 0.5. The ROC-AUC measures area under the entire precision-recall trade-off curve, while F1 measures performance at the specific operating point. This discrepancy is common in imbalanced classification problems.

6.2.3 Comparison with Baseline

A simple majority-class baseline (always predicting 'clean') would achieve approximately 71.6% accuracy given the 2.5:1 class ratio. The model improves accuracy by only 2.6 percentage points. However, the majority

baseline has zero recall for bugs — it would catch no bugs whatsoever. The model's recall of 0.826 represents a substantial improvement in the metric that matters most for this application.

6.2.4 Training Dynamics

The divergence between training and validation loss beginning at epoch 4 indicates mild overfitting. This is expected given that the training set contains only 27,758 samples — small by transformer fine-tuning standards. Mitigations applied in this project include layer freezing (bottom 6 layers) and dropout. Future work could explore additional regularisation strategies such as label smoothing or data augmentation via code transformation.

6.3 Explainability Analysis

Attention visualization reveals interpretable patterns in the model's decision-making. For buggy code samples, the model consistently attends to:

- ❖ Division operations and denominators (potential ZeroDivisionError patterns)
- ❖ Array/list index expressions (potential IndexError patterns)
- ❖ Null/None checks or their absence (potential NullPointerException equivalents)
- ❖ Loop boundary conditions (off-by-one error patterns)

These patterns align with the types of bugs present in the Devign and GitHub training data, suggesting the model has learned genuine code quality signals rather than superficial correlations.

SHAP analysis confirms the attention findings for most samples. For a small subset of predictions, SHAP and attention disagree, consistent with the theoretical finding that attention weights are not guaranteed measures of feature importance. For production use, SHAP would provide stronger theoretical grounding despite higher computational cost.

6.4 Limitations

- ❖ **Language bias:** The majority of labelled buggy samples come from C code (Devign), while clean samples are predominantly Python (CodeSearchNet). The model may perform differently on pure Python bug detection than benchmarks suggest.
- ❖ **Bug type coverage:** The training data reflects vulnerability patterns in open-source system software (C). Application-level bugs, logic errors, and performance issues common in Python data science code are underrepresented.
- ❖ **Short code snippets:** The 512-token limit truncates longer functions. For large files or classes, only the first ~200-300 lines of code are evaluated.

- ❖ **Attention as explanation:** Attention weights are an approximation of explanation, not a causal mechanism. They should be interpreted as weak guidance, not definitive attribution.
- ❖ **No fix suggestions:** The current model only detects bugs; it does not suggest corrections. A seq2seq model (e.g., CodeT5+) would be needed for automated repair.

7. Future Work

Several extensions would improve the system's capability and practical utility:

7.1 Model Improvements

- ❖ Fine-tune GraphCodeBERT incorporating data flow graph structure for improved C vulnerability detection
- ❖ Train a CodeT5+ seq2seq model to generate fix suggestions alongside detection
- ❖ Explore multi-class classification (e.g., null dereference, buffer overflow, off-by-one, logic error) for more actionable bug categorization
- ❖ Apply label smoothing and code augmentation (variable renaming, dead code insertion) to reduce overfitting
- ❖ Train on BigVul or ReVeal datasets for larger-scale vulnerability coverage

7.2 Explainability

- ❖ Replace attention-based explanation with Integrated Gradients for theoretically grounded token attribution
- ❖ Implement layer-wise relevance propagation (LRP) for finer-grained analysis
- ❖ Add per-bug-type explanation templates that describe the specific vulnerability pattern in natural language

7.3 Deployment

- ❖ Package the model as a VS Code extension for in-editor real-time feedback
- ❖ Implement a GitHub Actions CI/CD integration that runs code review on pull requests automatically
- ❖ Deploy as a REST API with Docker for integration into existing CI pipelines
- ❖ Add support for additional programming languages (Java, JavaScript) using the full CodeSearchNet corpus

7.4 Evaluation

- ❖ Benchmark against commercial tools (SonarQube, CodeClimate) on standardized bug datasets
- ❖ Conduct a user study with software developers to evaluate practical utility and explanation quality
- ❖ Evaluate on out-of-distribution code (e.g., proprietary enterprise codebases) to measure real-world generalization

8. Conclusion

This project successfully designed, implemented, and deployed an Intelligent Code Review Assistant using CodeBERT fine-tuning on a multi-source labelled dataset of 39,656 code samples. The system addresses a genuine software engineering challenge, scalable, automated bug detection using state-of-the-art deep learning techniques from NLP.

The four-notebook pipeline demonstrates the complete data science workflow: from raw data collection via HuggingFace and GitHub APIs, through preprocessing and tokenization, to GPU-accelerated transformer fine-tuning, evaluation with multiple metrics, and deployment as an interactive Streamlit web application. The hybrid Jupyter/Colab environment proved practical for managing local data preparation alongside cloud GPU training within a four-week timeline.

The model achieves 74.2% accuracy, 0.826 recall, and 0.852 ROC-AUC — meaningfully better than a random or majority-class baseline for the practically critical task of bug recall. The attention-based explainability system provides interpretable token-level highlights that help developers understand why a particular code segment was flagged, addressing a key barrier to adoption of automated review tools.

The project's identified limitations, language bias from training data composition, attention's theoretical limitations as an explanation method, and absence of fix generation provide a clear roadmap for future improvement. Taken together, the work demonstrates that transformer-based code analysis is practical, deployable, and useful in the four-week timeframe of an individual internship project.

Appendix

Appendix A — Project File Structure

```
intelligent-code-review-assistant/
├── data/
│   ├── raw/                ← CodeSearchNet, GitHub PRs
│   └── processed/          ← train.pt, val.pt, test.pt, class_weights
├── models/
│   └── codebert_classifier/
│       ├── model.safetensors ← Trained weights (475 MB)
│       ├── classifier_head.pt ← Classification head weights
│       └── tokenizer_config.json ← Tokeniser config
├── notebooks/
│   ├── 01_data_collection_eda.ipynb    ← Jupyter (local)
│   ├── 02_preprocessing_tokenization.ipynb ← Jupyter (local)
│   ├── 03_model_training.ipynb        ← Google Colab (T4 GPU)
│   └── 04_evaluation.ipynb            ← Google Colab (T4 GPU)
├── app/
│   └── app.py                        ← Streamlit dashboard
├── reports/
│   ├── figures/                    ← 20 saved visualisation figures
│   ├── training_results.pkl        ← History + metrics + predictions
│   └── classification_report.csv    ← Per-class classification report
├── requirements.txt
└── README.md
```

Appendix B — Complete List of Saved Figures

Filename	Notebook	Description
01_label_distribution.png	NB01	Label count and percentage (clean vs. buggy)
02_samples_per_source.png	NB01	Sample count by data source

Filename	Notebook	Description
03_code_length_distribution.png	NB01	Code length histogram — clean vs. buggy
04_language_breakdown.png	NB01	Samples by language and label
05_boxplot_code_length.png	NB01	Code length boxplot by label
06_correlation_heatmap.png	NB01	Feature correlation matrix
07_token_length_distribution.png	NB02	Token lengths before truncation
08_token_distribution_splits.png	NB02	Real token distribution per split
09_class_balance_splits.png	NB02	Class balance across train/val/test
10_training_curves.png	NB03	Loss, F1, precision/recall, ROC-AUC over epochs
11_test_evaluation.png	NB03	Confusion matrix and ROC curve on test set
12_confidence_distribution.png	NB04	Model confidence distribution by true label
13_error_analysis.png	NB04	False positive and false negative analysis
14_attention_true_positive.png	NB04	Attention — correctly flagged bug
15_attention_false_negative.png	NB04	Attention — missed bug
16_attention_true_negative.png	NB04	Attention — correctly identified clean code
17_shap_bar.png	NB04	SHAP summary bar chart — top 20 tokens
18_shap_waterfall_buggy.png	NB04	SHAP waterfall — buggy sample
19_shap_waterfall_clean.png	NB04	SHAP waterfall — clean sample
20_final_metrics_summary.png	NB04	Final metrics summary dashboard

Table A.1: Complete list of saved figures with source notebook

Appendix C — Running the Project

C.1 Environment Setup

```
git clone https://github.com/YOUR_USERNAME/intelligent-code-review-assistant
cd intelligent-code-review-assistant
python -m venv venv
venv\Scripts\activate      # Windows
```

```
pip install -r requirements.txt
```

C.2 Run Notebooks (in order)

```
jupyter notebook notebooks/01_data_collection_eda.ipynb
jupyter notebook notebooks/02_preprocessing_tokenization.ipynb
# Upload data/processed/ to Google Drive, then open in Colab:
# notebooks/03_model_training.ipynb
# notebooks/04_evaluation.ipynb
```

C.3 Launch Dashboard

```
streamlit run app/app.py
```

End of the Report.
